

Table des matières

1	Introduction	3
2	Premier programme en C	3
3	Variables et Types de Données	3
3.1	Définition et syntaxe	3
3.2	Conversion de type, cast	4
4	Fonctions et procédures	5
4.1	Définition et exemples	5
4.2	Effet de bord	6
5	Pointeurs et tableaux	7
5.1	Principe du pointeur	7
5.2	Accès aux éléments d'un tableau via un pointeur	7
5.3	Parcours d'un tableau avec un pointeur	8
5.4	Différence entre un tableau et un pointeur	8
6	Structures de contrôles	8
6.1	if : Conditionnelle	8
6.2	switch-case : Alternative aux if-else	10
6.3	for : Boucle à itération contrôlée	11
6.4	while : Boucle à condition d'arrêt	11
6.5	do while : Boucle exécutée au moins une fois	12
7	Manipulations Binaires	12
8	Définition et Utilisation d'une Structure	14
8.1	Exemple du Buffer Circulaire	15
9	Passage des Arguments dans des Fonctions	16
9.1	Passage par Valeur	16
9.2	Passage par pointeurs	16
9.3	Passage d'un Tableau	17

TABLE DES MATIÈRES

TABLE DES MATIÈRES

9.4	Passage d'une Structure	18
9.5	Le mot-clé <code>const</code>	19
9.5.1	Utilisation de <code>const</code> pour les arguments	19
9.5.2	Utilisation de <code>const</code> pour les variables globales	19
9.5.3	Const et tableaux	19
10	Le fonctionnement de <code>#include</code> et l'utilisation de <code>#ifndef</code>	20
10.1	Utilisation de <code>#ifndef</code> pour éviter les inclusions multiples	20
10.2	Un exemple	20
11	Macros et préprocesseur	21
11.1	Introduction	21
11.2	Macro et multiplateforme : un exemple	21
11.3	Multithread en C	22
11.4	Les mots-clés <code>extern</code> et <code>static</code>	22
12	Exemples sur Arduino	22
12.1	Communication Série, I2C, SPI et interruptions	22

1 Introduction

Le langage C est très utilisé pour programmer les microcontrôleurs et des systèmes embarqués. Ce tutoriel présente les bases du C avec des exemples et des erreurs courantes sous GCC.

2 Premier programme en C

Le langage C requiert une fonction principale appelée `main`, qui sert de point d'entrée au programme. Sans cette fonction, le programme ne pourra pas être exécuté.

```
#include <stdio.h>

int main() {
    printf("Bonjour, Polytech !\n");
    return 0;
}
```

Compilation avec GCC :

```
gcc main.c -o main
./main
```

Sortie attendue :

```
Bonjour, Polytech !
```

Définition 1 (Langage compilé et compilateur). Le langage C est un langage compilé, ce qui signifie que tout son code source est traduit en code machine par un compilateur avant d'être exécuté, garantissant ainsi des performances optimales. À l'inverse, un langage interprété exécute directement chaque ligne du code via un interpréteur, ce qui le rend plus flexible, mais souvent moins rapide.

Un compilateur est un programme très bien conçu et il est très raisonnable de lui faire confiance, notamment sur les optimisations. À savoir, tous les compilateurs ne se comportent pas exactement de la même manière. GCC est l'un des compilateurs les plus couramment utilisés, mais d'autres comme Clang ou MSVC peuvent avoir des différences de gestion des erreurs et d'optimisation.

Bonne pratique 1 (Ecrire du magnifique code source en C). TODO

3 Variables et Types de Données

3.1 Définition et syntaxe

```
#include <stdio.h>

int main() {
    int capteur = 25;
```

```
float tension = 3.3;

printf("Capteur: %d °C\n", capteur);
printf("Tension: %.2f V\n", tension);

return 0;
}
```

Erreur de conception possible avec format incorrect par rapport au type attendu :

```
printf("Tension: %d V\n", tension); // Avertissement !
```

Warning GCC :

```
warning: format '%d' expects argument of type 'int', but argument 2 has type 'float'
```

Correction : Utiliser %f au lieu de %d dans printf.

Définition 2 (Langage à typage fort). Le langage C est un langage à typage fort, ce qui signifie que chaque variable a un type bien défini qui ne peut pas être changé dynamiquement. Cela force le programmeur à être rigoureux dans ses déclarations et conversions.

Remarque 1 (Spécificateurs de format). Voici quelques formats courants pour afficher des valeurs :

- %d : entier signé (int)
- %u : entier non signé (unsigned int)
- %f : nombre flottant
- %x, %X : affichage en hexadécimal
- %p : pointeur
- %c : caractère unique
- %s : chaîne de caractères

3.2 Conversion de type, cast

Le casting permet de convertir un type en un autre.

```
#include <stdio.h>

int main() {
    int a = 5;
    int b = 2;
    float result = a / b; // pas de warning !
    printf("Résultat: %f\n", result); // Va afficher 2.0000, pas 2.5 !
    return 0;
}
```

L'exemple au-dessus est très subtil, car l'opération de division qui sera utilisée sera celle de la division entière, et non pas la division "standard". Un outil d'analyse de code tel que `clang-tidy` est capable d'identifier ce type de comportement.

Correction : Utiliser un cast explicite sur l'un des opérandes.

```
float result = (float)a / b;
```

Toutefois, une mauvaise conversion peut entraîner des erreurs difficiles à détecter, notamment sur des conversions entiers flottants, dans un sens comme dans l'autre.

4 Fonctions et procédures

4.1 Définition et exemples

Définition 3 (Fonction, procédure et signature). En C, une **fonction** est un bloc de code réutilisable qui peut retourner une valeur, tandis qu'une **procédure** est simplement une fonction qui ne retourne pas de valeur (utilisant le type `void`).

- **Fonction** : prend des paramètres en entrée, effectue des opérations et retourne un résultat.
- **Procédure** : similaire à une fonction, mais sans retour de valeur explicite.
- **Signature d'une fonction** : définit son type de retour, son nom et ses paramètres.

Exemple de signature :

```
int somme(int a, int b);
```

Ici, `somme` est une fonction prenant deux entiers en paramètre (*a* et *b*) et retournant un entier.

```
#include <stdio.h>

int somme(int a, int b) {
    return a + b;
}

int main() {
    int resultat = somme(3, 4);
    printf("Résultat : %d\n", resultat); // Affiche 7
    return 0;
}
```

Exemple de fonction retournant une valeur

```
#include <stdio.h>

void afficherMessage() {
    printf("Bonjour, Polytech !\n");
}

int main() {
    afficherMessage();
    return 0;
}
```

Exemple de procédure (fonction sans retour)

Dans cet exemple, la fonction `afficherMessage()` ne retourne aucune valeur et sert uniquement à afficher un message.

Les fonctions et procédures permettent de structurer le code, d'améliorer sa lisibilité et de favoriser la réutilisation.

4.2 Effet de bord

Définition 4 (Effet de bord). Un effet de bord se produit lorsqu'une fonction ou une procédure modifie une variable extérieure à son propre scope ou produit un résultat observable en dehors de son retour de valeur (comme l'affichage à l'écran ou l'écriture dans un fichier).

```
#include <stdio.h>

int compteur = 0; // Variable globale

void incrementer() {
    compteur++; // Modifie une variable externe
}

int main() {
    incrementer();
    printf("Compteur : %d\n", compteur); // Affichera 1
    return 0;
}
```

Exemple d'effet de bord involontaire

Ici, la fonction `incrementer()` modifie directement la variable `compteur`, ce qui peut entraîner des comportements inattendus dans un programme complexe.

Bonne pratique 2. Minimiser les effets de bord en évitant d'utiliser des variables globales et en favorisant le passage d'arguments et le retour de valeurs. L'utilisation du mot-clé `const` peut également aider à éviter les modifications accidentelles.

5 Pointeurs et tableaux

En C, les tableaux et les pointeurs sont étroitement liés. Un tableau est en réalité une séquence contiguë d'éléments en mémoire, et son nom agit comme un pointeur constant vers son premier élément.

5.1 Principe du pointeur

Un **pointeur** est une variable qui stocke l'adresse mémoire d'une autre variable. Il permet d'accéder directement à une zone mémoire et de modifier son contenu.

```
#include <stdio.h>

int main() {
    int x = 10;
    int *ptr = &x; // ptr stocke l'adresse de x, l'opérateur & permet d'obtenir l'adresse

    printf("Valeur de x : %d\n", x);
    printf("Adresse de x : %p\n", &x);
    printf("Valeur stockée à l'adresse contenue dans ptr : %d\n", *ptr);

    *ptr = 20; // Modification de x via le pointeur
    printf("Nouvelle valeur de x : %d\n", x);

    return 0;
}
```

Explications :

- `int *ptr;` déclare un pointeur vers un entier.
- `ptr = &x;` affecte l'adresse de `x` à `ptr`.
- `*ptr` (déréférencement) permet d'accéder à la valeur pointée et de la modifier.

5.2 Accès aux éléments d'un tableau via un pointeur

En C, les tableaux et les pointeurs sont étroitement liés. Un tableau est en réalité une séquence contiguë d'éléments en mémoire, et son nom agit comme un pointeur constant vers son premier élément. L'utilisation d'un pointeur permet d'accéder aux éléments d'un tableau de manière équivalente à l'indexation classique.

```
#include <stdio.h>

int main() {
    int tableau[] = {10, 20, 30, 40};
    int *ptr = tableau; // Pointeur sur le premier élément

    printf("Premier élément : %d\n", *ptr);
    printf("Deuxième élément : %d\n", *(ptr + 1)); // Équivalent à tableau[1]

    return 0;
}
```

5.3 Parcours d'un tableau avec un pointeur

Les pointeurs permettent de parcourir un tableau sans utiliser d'index, en incrémentant simplement l'adresse stockée dans le pointeur.

```
#include <stdio.h>

int main() {
    int tableau[] = {5, 10, 15, 20, 25};
    int *ptr = tableau;

    while (ptr < tableau + 5) { // Tant que l'on ne dépasse pas la fin du tableau
        printf("%d ", *ptr);
        ptr++; // Déplacement du pointeur
    }
    printf("\n");

    return 0;
}
```

5.4 Différence entre un tableau et un pointeur

Bien que le nom d'un tableau puisse être utilisé comme un pointeur, il existe des différences importantes :

- Le nom du tableau est une adresse constante, tandis qu'un pointeur est une variable qui peut être modifiée.
- On ne peut pas réaffecter un tableau, mais on peut modifier un pointeur pour qu'il pointe vers une autre adresse.

```
int tableau[] = {1, 2, 3};
int *ptr = tableau;

ptr = ptr + 1; // OK, déplacement du pointeur
tableau = tableau + 1; // Erreur, un tableau est une adresse constante
```

GCC sortira alors l'erreur suivante :

```
error: array type 'int[3]' is not assignable
```

6 Structures de contrôles

Les structures de contrôle en C permettent de modifier le flux d'exécution d'un programme en fonction de conditions ou de répétitions d'instructions. Les principales structures de contrôle sont :

6.1 if : Conditionnelle

L'instruction `if` permet d'exécuter un bloc de code uniquement si une condition est vraie.

```
#include <stdio.h>

int main() {
```



```
int temperature = 30;

if (temperature > 25) {
    printf("Il fait chaud !\n");
} else {
    printf("La température est agréable.\n");
}

return 0;
}
```

Remarque 2 (Valeur et vérité). *Une valeur non-nulle ou non-vide est considérée comme vraie*

```
#include <stdio.h>

int main()
{
    if(1) {printf("J'");}
    if(NULL) {printf("n");}
    if(-1) {printf("aime ");}
    if(0) {printf("pas");}
    if(2.5) {printf("les ");}
    if("fruit") {printf("bananes");}
    return 0;
}
```

L'exemple sortira "J'aime les bananes".

Bonne pratique 3 (Le cas de la comparaison des nombres flottants). Les nombres flottants, représentés selon la norme IEEE-754 en base 2 (binaire), sont des approximations des nombres réels. Par conséquent, certains nombres décimaux (par exemple, 0.1) ne peuvent pas être représentés exactement, ce qui introduit des erreurs d'arrondi. Ces erreurs, lorsqu'elles s'accumulent au cours d'opérations arithmétiques, font que deux valeurs affichées de manière identique peuvent légèrement différer en mémoire.

Pour cette raison, l'utilisation de l'opérateur `==` pour comparer directement des flottants est peu fiable. Il est recommandé de vérifier que la différence absolue entre deux valeurs est inférieure à un seuil *epsilon* adapté au contexte. Par exemple :

```
#include <stdio.h>
#include <math.h>

#define EPSILON 0.0001

int main()
{
    double a = 0.1 + 0.2;
    double b = 0.3;
    if (fabs(a - b) < EPSILON)
    {
        printf("Les valeurs sont considérées comme égales.\n");
    }
    else
    {
        printf("Les valeurs sont différentes.\n");
    }
    printf("Et voilà ce que dit l'opérateur d'égalité : %d", a==b);

    return 0;
}
```

Et la sortie qui sera :

```
Les valeurs sont considérées comme égales.
Et voilà ce que dit l'opérateur d'égalité : 0
```

6.2 switch-case : Alternative aux if-else

L'instruction switch-case permet d'exécuter différents blocs de code en fonction de la valeur d'une variable. Elle est souvent utilisée lorsque plusieurs conditions doivent être testées sur une même variable.

```
#include <stdio.h>

int main() {
    int jour = 3;

    switch (jour) {
        case 1:
            printf("Lundi\n");
            break;
        case 2:
            printf("Mardi\n");
            break;
        case 3:
            printf("Mercredi\n");
            break;
        case 4:
            printf("Jeudi\n");
    }
```

```

        break;
    case 5:
        printf("Vendredi\n");
        break;
    default:
        printf("Week-end !\n");
        break;
}

return 0;
}

```

Explication :

- Chaque `case` correspond à une valeur possible de la variable `jour`.
- L'instruction `break` est utilisée pour empêcher l'exécution des autres cas après un match.
- Le `default` est exécuté si aucune des valeurs ne correspond.

On notera que dans cet exemple, toute valeur qui n'est 1,2,3,4 ou 5 déclenchera la clause par défaut.

Le `switch-case` est souvent plus lisible et plus efficace qu'une série de `if-else` lorsque l'on compare une seule variable à plusieurs valeurs possibles.

Bonne pratique 4. Toujours finir un `switch-case` par une clause par défaut : cela permet d'éviter d'avoir des bugs dus à des valeurs inattendues.

6.3 *for* : Boucle à itération contrôlée

La boucle `for` est utilisée lorsque le nombre d'itérations est connu à l'avance.

```

#include <stdio.h>

int main() {
    for (int i = 0; i < 5; i++) {
        printf("Iteration %d\n", i);
    }
    return 0;
}

```

6.4 *while* : Boucle à condition d'arrêt

La boucle `while` exécute un bloc tant qu'une condition est vraie.

```

#include <stdio.h>

int main() {
    int compteur = 0;

    while (compteur < 3) {
        printf("Compteur : %d\n", compteur);
    }
}

```

```
    compteur++;  
}  
  
return 0;  
}
```

6.5 `do while` : Boucle exécutée au moins une fois

La boucle `do while` garantit au moins une exécution du bloc avant de tester la condition.

```
#include <stdio.h>  
  
int main() {  
    int nombre = 5;  
  
    do {  
        printf("Valeur : %d\n", nombre);  
        nombre--;  
    } while (nombre > 0);  
  
    return 0;  
}
```

7 Manipulations Binaires

Les manipulations binaires sont essentielles en programmation bas niveau, notamment pour configurer des registres et gérer des entrées/sorties. En C, les opérateurs binaires permettent de manipuler directement les bits d'une variable.

Par exemple :

- **ET bit à bit (&)** : permet d'extraire certains bits en appliquant un masque.
- **OU bit à bit (|)** : active des bits spécifiques sans modifier les autres.
- **XOR (OU exclusif, ^)** : inverse certains bits tout en laissant les autres inchangés (l'un ou l'autre, mais pas les deux).
- **Décalages (<<, >>)** : déplacent les bits vers la gauche ou la droite, utile pour des opérations de multiplication ou division rapide par puissances de 2.
- **Négation bit à bit (~)** : inverse tous les bits d'un nombre, transformant chaque 0 en 1 et chaque 1 en 0. Cette opération est utilisée notamment pour créer des masques et pour obtenir le complément à un d'un nombre.

Rappel 1 (Opérateurs et logique booléenne). En logique booléenne, les opérations suivantes sont fondamentales :

- **AND** : Retourne 1 si les deux opérandes sont 1, sinon 0.
- **OR** : Retourne 1 si au moins l'un des opérandes est 1, sinon 0.
- **XOR** : Retourne 1 si un seul des opérandes est 1, sinon 0.
- **NOT** : Inverse la valeur d'entrée (1 devient 0 et vice-versa).

Dans l'exemple suivant, `registre` & `masque` permet de conserver uniquement les 4 bits de poids faible du registre tout en mettant les autres à zéro. Ces techniques sont fréquemment utilisées pour configurer des ports d'un microcontrôleur ou extraire des informations précises d'un signal binaire.

```
#include <stdio.h>

int main() {
    unsigned char registre = 0b11001100;
    unsigned char masque = 0b00001111;
    unsigned char resultat = registre & masque;
    printf("Résultat après masquage : 0x%X\n", resultat);
    return 0;
}
```

Rappel 2 (Représentation des nombres). En informatique, un même nombre peut être représenté sous différentes bases :

- **Binaire (base 2)** : Utilise uniquement les chiffres 0 et 1. C'est la représentation native des nombres en informatique.
- **Octal (base 8)** : Utilise les chiffres de 0 à 7. Historiquement utilisé pour représenter les groupes de 3 bits.
- **Décimal (base 10)** : Système de numération le plus couramment utilisé par les humains.
- **Hexadécimal (base 16)** : Utilise les chiffres de 0 à 9 et les lettres A à F. Très utilisé en programmation pour représenter des valeurs binaires de manière plus compacte.

En C, les notations suivantes permettent de déclarer des nombres dans différentes bases :

```
int decimal = 42;           // Base 10
int binaire = 0b101010;    // Base 2 (C99+)
int octal = 052;           // Base 8 (commence par 0)
int hexa = 0x2A;           // Base 16 (commence par 0x)
```

Bonne pratique 5. Voici quelques suggestions par rapport aux différentes bases utilisées en C :

- Utiliser l'hexadécimal pour représenter des adresses mémoire et des valeurs binaires de grande taille.
- Éviter l'octal, qui peut prêter à confusion car les nombres commencent par un zéro.
- Toujours ajouter des commentaires explicites pour clarifier les valeurs utilisées dans différentes bases.

Définition 5 (Bit de poids fort, poids faible, big endian et small endian). Commençons par la différence entre bit de poids fort et bit de poids faible :

- **Bit de Poids Fort** (Most Significant Bit ou MSB) : Le bit de poids fort est le bit qui a la valeur la plus significative dans une séquence de bits. C'est généralement le bit le plus à gauche d'un nombre binaire, et il détermine la plus grande partie de la valeur du nombre. Par exemple, dans le nombre binaire 1011, le bit de poids fort est le 1 à l'extrême gauche.

- **Bit de Poids Faible** (Least Significant Bit ou LSB) : Le bit de poids faible est le bit qui a la valeur la moins significative dans une séquence de bits. C'est généralement le bit le plus à droite d'un nombre binaire. Dans le même exemple 1011, le bit de poids faible est le 1 à l'extrême droite.

En ce qui concerne les différences d'ordonnement,

- **Big Endian** : Le format `big endian` est un ordre de stockage des octets dans lequel le bit de poids fort (le premier octet) est stocké en premier, à l'adresse mémoire la plus basse. En d'autres termes, le premier octet correspond à la partie la plus significative du nombre. Par exemple, le nombre `0x12345678` serait stocké en mémoire de la manière suivante :

Mémoire : `0x12 0x34 0x56 0x78`

Par exemple, les DSP SHARC ou les Motorola 68000 sont Big Endian.

- **Little Endian** : Le format `little endian` est un ordre de stockage des octets dans lequel le bit de poids faible (le dernier octet) est stocké en premier, à l'adresse mémoire la plus basse. Le premier octet correspond à la partie la moins significative du nombre. Par exemple, le nombre `0x12345678` serait stocké en mémoire de la manière suivante :

Mémoire : `0x78 0x56 0x34 0x12`

Les processeurs Intel sont Little Endian.

Ces différences d'ordonnement peuvent affecter la manière dont les données sont lues ou écrites entre différents systèmes ou architectures. Certaines architectures utilisent le `big endian`, tandis que d'autres utilisent le `little endian`. La conversion entre les deux formats est essentielle lorsqu'on échange des données entre systèmes utilisant des architectures différentes.

8 Définition et Utilisation d'une Structure

```
#include <stdio.h>

typedef struct {
    float tension;
    float courant;
    float puissance;
} Mesure;

void calculer_puissance(Mesure *m) {
    m->puissance = m->tension * m->courant;
}

int main() {
    Mesure capteur = {12.0, 2.0, 0.0};
    calculer_puissance(&capteur);
    printf("Puissance : %.2f W\n", capteur.puissance);
    return 0;
}
```

L'utilisation de `typedef` dans la définition d'une structure en C permet de simplifier son utilisation en évitant d'avoir à répéter le mot-clé `struct` à chaque déclaration de variable.

Pourquoi utiliser typedef? Sans typedef, la déclaration d'une variable de type `Mesure` nécessiterait d'écrire explicitement `struct` :

```
struct Measure {  
    float tension;  
    float courant;  
    float puissance;  
};  
  
// Déclaration d'une variable  
struct Measure capteur; // Obligatoire sans typedef
```

Avec typedef, on peut directement utiliser `Mesure` comme un type personnalisé, ce qui allège la syntaxe :

```
typedef struct {  
    float tension;  
    float courant;  
    float puissance;  
} Measure;  
  
// Déclaration d'une variable  
Measure capteur; // Plus simple
```

Pourquoi l'absence de typedef peut poser problème ?

1. **Lisibilité du code** : Répéter `struct` alourdit la déclaration et rend le code moins clair, surtout lorsque les structures sont utilisées fréquemment.
2. **Erreurs de maintenance** : Dans de grands projets, devoir toujours écrire `struct` peut être source d'erreurs et de confusion.
3. **Compatibilité avec d'autres types** : typedef permet de masquer l'implémentation interne et de changer la définition sans impacter le reste du code.

Dans un contexte système embarqué, où l'on manipule souvent des structures pour stocker des données de capteurs ou des configurations, typedef améliore considérablement la lisibilité et la maintenabilité du code.

Différence entre . et -> Suivant si l'on a à faire avec un objet ou un pointeur, on n'utilisera pas la même syntaxe pour accéder aux éléments de la structure :

```
Mesure m;  
m.tension = 12.0; // Accès direct  
  
Mesure *ptr = &m;  
ptr->tension = 12.0; // Accès via pointeur
```

8.1 Exemple du Buffer Circulaire

Un **buffer circulaire** est une structure de données qui utilise un tableau fixe et deux indices (tête et queue) pour gérer des données en FIFO (First In, First Out). Il est très utilisé en système embarqué pour la gestion des flux de données, comme la communication série.

TODO

Ce code met en œuvre un buffer circulaire capable de stocker un nombre limité d'éléments, avec une gestion propre de l'insertion et de la suppression.

9 Passage des Arguments dans des Fonctions

9.1 Passage par Valeur

Lorsque vous passez un argument par valeur, une copie est créée et toute modification dans la fonction ne modifie pas la variable originale.

```
#include <stdio.h>

int carre(int n) {
    n = n * n;
    return n;
}

int main() {
    int nombre = 5;
    printf("Carré de %d : %d\n", nombre, carre(nombre));
    printf("Valeur originale après appel : %d\n", nombre); // Reste 5
    return 0;
}
```

Exemple : Carré d'un nombre

Explication :

- La variable `nombre` reste inchangée, car `n` est une copie.

Rappel 3 (Portée d'une variable). En C, la **portée d'une variable** définit la partie du programme où cette variable est accessible. On distingue principalement :

- **Portée locale** : Une variable déclarée à l'intérieur d'une fonction n'est accessible que dans cette fonction.
- **Portée globale** : Une variable déclarée en dehors de toute fonction est accessible depuis tout le programme, sauf si elle est déclarée `static`, limitant ainsi son accès au fichier courant.

Lien avec le passage par valeur

Lorsqu'un argument est passé **par valeur** à une fonction, une copie de cette variable est créée dans la mémoire locale de la fonction. Toute modification de cette copie n'affecte pas la variable d'origine, parce que la copie a une portée plus locale en plus d'être une copie.

9.2 Passage par pointeurs

Lorsque vous passez un pointeur, la fonction peut directement modifier la variable originale.

```
#include <stdio.h>

void echanger(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;
    printf("Avant échange : x = %d, y = %d\n", x, y);

    echanger(&x, &y);

    printf("Après échange : x = %d, y = %d\n", x, y);
    return 0;
}
```

Exemple : Échange de deux nombres

Explication :

— *a et *b modifient directement x et y.

9.3 Passage d'un Tableau

En C, lorsqu'un tableau est passé en argument à une fonction, ce n'est pas une copie du tableau qui est transmise, mais un **pointeur** vers son premier élément. Cela signifie que toute modification effectuée dans la fonction affectera directement le tableau original.

```
#include <stdio.h>

void doublerValeurs(int tab[], int taille) {
    for (int i = 0; i < taille; i++) {
        tab[i] *= 2;
    }
}

int main() {
    int valeurs[] = {1, 2, 3, 4};
    int taille = sizeof(valeurs) / sizeof(valeurs[0]);

    printf("Avant : ");
    for (int i = 0; i < taille; i++) {
        printf("%d ", valeurs[i]);
    }
    printf("\n");

    doublerValeurs(valeurs, taille);
}
```

```
printf("Après : ");  
for (int i = 0; i < taille; i++) {  
    printf("%d ", valeurs[i]);  
}  
printf("\n");  
  
return 0;  
}
```

Exemple : Modification d'un tableau**Explication :**

- Les modifications du tableau sont conservées après l'appel.

9.4 Passage d'une Structure

Lorsque vous passez une structure par valeur, une copie est créée. Si vous utilisez un pointeur, la modification affectera l'original pointé.

```
#include <stdio.h>  
  
typedef struct {  
    float tension;  
    float courant;  
} Mesure;  
  
void modifierMesure(Mesure *m) {  
    m->tension += 1.0;  
    m->courant *= 2;  
}  
  
int main() {  
    Mesure capteur = {5.0, 1.5};  
  
    printf("Avant : Tension = %.1f V, Courant = %.1f A\n", capteur.tension,  
        ↪ capteur.courant);  
  
    modifierMesure(&capteur);  
  
    printf("Après : Tension = %.1f V, Courant = %.1f A\n", capteur.tension,  
        ↪ capteur.courant);  
  
    return 0;  
}
```

Exemple : Passage d'une structure par pointeur**Explication :**

- `m->tension` et `m->courant` modifient directement les valeurs de `capteur`.

9.5 Le mot-clé `const`

Le mot-clé `const` permet d'indiquer qu'une variable ou un argument ne doit pas être modifié, ce qui améliore la sûreté et l'optimisation du code.

9.5.1 Utilisation de `const` pour les arguments

Lorsque l'on passe une variable par référence avec un pointeur, il est parfois souhaitable de garantir que la fonction ne la modifiera pas. Cela peut être assuré avec `const` :

```
#include <stdio.h>

void afficher(const int *val) {
    printf("Valeur : %d\n", *val);
    // *val = 10; // Erreur : tentative de modification d'un argument constant
}

int main() {
    int nombre = 42;
    afficher(&nombre);
    return 0;
}
```

Ici, le pointeur `const int *val` indique que la valeur pointée ne peut pas être modifiée dans la fonction.

9.5.2 Utilisation de `const` pour les variables globales

Les variables globales définies avec `const` ne peuvent pas être modifiées accidentellement, ce qui évite des erreurs dans les grands programmes :

```
const float PI = 3.14159;

int main() {
    // PI = 3.14; // Erreur : PI est une constante
    printf("Valeur de PI : %.5f\n", PI);
    return 0;
}
```

Ici, GCC sortira :

```
error: assignment of read-only variable 'PI'
```

9.5.3 Const et tableaux

`const` peut également être utilisé pour empêcher la modification d'un tableau passé à une fonction :

```
void afficherTableau(const int tab[], int taille) {
    for (int i = 0; i < taille; i++) {
        printf("%d ", tab[i]);
    }
}
```

```
printf("\n");
}
```

L'utilisation de `const` ici empêche la modification des éléments du tableau dans la fonction.

L'ajout de `const` améliore la lisibilité et la robustesse du code en garantissant que certaines valeurs restent inchangées, ce qui est essentiel dans le développement embarqué et mécatronique.

Remarque 3 (Bonne pratique). *Si une variable ou un argument ne doit pas changer, il doit constamment être mis `const` : cela permet d'éviter des bugs dès la conception !*

10 Le fonctionnement de #include et l'utilisation de #ifndef

En C, la directive `#include` permet d'inclure le contenu d'un fichier dans un autre avant la compilation. Le préprocesseur remplace chaque `#include` par le contenu du fichier spécifié, ce qui est essentiel pour structurer un programme en plusieurs fichiers et éviter la redondance.

10.1 Utilisation de #ifndef pour éviter les inclusions multiples

Lorsqu'un fichier header (.h) est inclus plusieurs fois, cela peut entraîner des erreurs de compilation dues à la redéfinition de types ou de fonctions. Pour éviter cela, on utilise des "gardiens d'inclusion" (*include guards*) avec `#ifndef` (*if not defined*) :

```
#ifndef MON_HEADER_H // Vérifie si MON_HEADER_H n'est pas défini
#define MON_HEADER_H // Définit MON_HEADER_H

// Contenu du fichier (déclarations de fonctions, structures, macros...)

#endif // Fin de la condition
```

Avec cette structure, si le fichier est inclus plusieurs fois, la première inclusion définit `MON_HEADER_H`, empêchant ainsi toute inclusion supplémentaire et évitant les erreurs de duplication.

10.2 Un exemple

```
#ifndef CALCUL_H
#define CALCUL_H

typedef struct {
    float tension;
    float courant;
} Mesure;

float calculer_puissance(Mesure m);

#endif
```

Fichier `calcul.h`

```
#include "calcul.h"

float calculer_puissance(Mesure m) {
    return m.tension * m.courant;
}
```

Fichier calcul.c

```
#include <stdio.h>
#include "calcul.h"

int main() {
    Mesure capteur = {12.0, 3.0};
    printf("Puissance : %.2f W\n", calculer_puissance(capteur));
    return 0;
}
```

Fichier main.c

Compilation :

```
gcc main.c calcul.c -o programme
./programme
```

Sortie attendue :

```
Puissance : 36.00 W
```

TODO ajouter les erreurs classiques d'include et leurs messages d'erreurs.

11 Macros et préprocesseur

11.1 Introduction

Le préprocesseur en C est un outil puissant qui effectue des transformations sur le code avant la compilation. Il permet notamment la définition de macros, l'inclusion de fichiers et la gestion de la compilation conditionnelle. Les macros permettent d'écrire du code plus générique et d'améliorer la portabilité des programmes.

11.2 Macro et multiplateforme : un exemple

Les macros peuvent être utilisées pour écrire un code compatible avec plusieurs plateformes. Par exemple, une macro conditionnelle peut adapter l'utilisation de certaines bibliothèques en fonction du système d'exploitation :

```
#ifdef _WIN32
    #define CLEAR_SCREEN "cls"
#else
    #define CLEAR_SCREEN "clear"
#endif
```

```
#include <stdlib.h>

void nettoyer_console() {
    system(CLEAR_SCREEN);
}
```

Ce code définit une macro `CLEAR_SCREEN` qui s'adapte automatiquement en fonction du système d'exploitation.

11.3 Multithread en C

Le multithreading permet d'exécuter plusieurs tâches en parallèle, ce qui est particulièrement utile pour les systèmes embarqués et les applications temps réel. En C, la bibliothèque `pthread` est souvent utilisée pour gérer les threads.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void *fonction_thread(void *arg) {
    printf("Thread exécuté !\n");
    return;
}

int main() {
    pthread_t thread;
    if (pthread_create(&thread, NULL, fonction_thread, NULL) != 0) {
        perror("Erreur de création du thread");
        return 1;
    }
    pthread_join(thread, NULL);
    return 0;
}
```

11.4 Les mots-clés `extern` et `static`

TODO

12 Exemples sur Arduino

12.1 Communication Série, I2C, SPI et interruptions

L'Arduino supporte plusieurs protocoles de communication pour interagir avec d'autres composants électroniques :

- **Port Série (UART)** : utilisé pour la communication entre l'Arduino et un ordinateur ou d'autres périphériques.
- **I2C** : permet la communication avec plusieurs périphériques sur un même bus.
- **SPI** : utilisé pour des échanges rapides de données avec des capteurs et des mémoires.

```
// Communication Série
```

```
void setup() {  
    Serial.begin(9600);  
}
```

```
void loop() {  
    Serial.println("Hello Arduino !");  
    delay(1000);  
}
```

```
// Communication I2C
```

```
#include <Wire.h>
```

```
void setup() {  
    Wire.begin();  
}
```

```
void loop() {  
    Wire.beginTransmission(0x3C); // Adresse de l'esclave  
    Wire.write(0x01); // Envoi de données  
    Wire.endTransmission();  
    delay(1000);  
}
```

```
// Communication SPI
```

```
#include <SPI.h>
```

```
void setup() {  
    SPI.begin();  
}
```

```
void loop() {  
    SPI.transfer(0xFF); // Envoi de données  
    delay(1000);  
}
```
